- *Microsoft.EntityFrameworkCore.InMemory*—This provider doesn't simulate a database. Instead, it stores objects directly in memory. It isn't a relational database as such, so it doesn't have all the features of a normal database. You can't execute SQL against it directly, and it won't enforce constraints, but it's fast. These limitations are large enough that Microsoft generally advise against using it. See http://mng.bz/e1E9.
- *Microsoft.EntityFrameworkCore.Sqlite*—SQLite is a relational database. It's limited in features compared with a database like SQL Server, but it's a true relational database, unlike the in-memory database provider. Normally a SQLite database is written to a file, but the provider includes an in-memory mode, in which the database stays in memory. This makes it much faster and easier to create and use for testing.

Unfortunately, EF Core migrations are tailored to a specific database, which means you can't run migrations created for SQL Server or PostreSQL against a SQLite database. It's possible to create multiple sets of migrations, as described in the documentation (http://mng.bz/pP15), but this can add a lot of complexity. Consequently, always use `EnsureCreated()` with SQLite tests, which creates the database without running migrations, as you'll see in listing 36.13.

Instead of storing data in a database on disk, both of these providers store data in memory, as shown in figure 36.2. This makes them fast and easy to create and tear down, which allows you to create a new database for every test to ensure that your tests stay isolated from one another.

